Greedy Design Paradigm

Solutions and C programs for Questions 1 - 10

Every answer has the same structure: the chosen **input representation**, the **algorithm** with a short correctness argument, the **complexity analysis**, a complete **C program** (it compiles with `gcc -O2` and reads from standard input) and a **sample run** produced by that exact program.

How the programs were validated

- Q1 was compared with an exhaustive search over all slot assignments (300 random cases, 0 mismatches).
- Q3, Q4, Q7, Q8 contain their own exhaustive / brute-force checker that prints PASS or FAIL; randomised runs gave no FAIL (300, 100, 500, 500 cases).
- Q5 was checked against a fixed-point brute force and the two methods against each other (500 cases). Q6 was checked against exhaustive permutation search (400 cases).
- Q9 (Hu-Tucker) was compared with an $O(n^3)$ interval DP on 3000 random inputs, including many with tied weights: 0 mismatches.
- Q2 verifies prefix-freeness and that the cost equals the sum of merged weights. Q10 checks that the output is a valid superstring and compares with the exact optimum for n <= 8.

Assumptions made where the question is open to interpretation

- **Q1:** weight is consumed in unit slots (time advances by 1 per unit of weight), a unit eaten in slot t is worth $v_i/w_i - \lambda_i t$. Partial amounts of an item are allowed; use a smaller unit to approximate real-valued fractions.
- **Q3:** one unit of fuel per unit of distance, unlimited tank, station i is at distance d_i from the origin and gives f_i fuel.
- **Q6:** 'distance K apart' means two equal characters have index difference at least K.
- **Q8:** meetings are half-open, so a meeting ending at time 10 and another starting at 10 can share a room.

Q1. Fractional Knapsack with Deterioration Rate

Problem. Items with value v_i, weight w_i, decay rate $\lambda_i > 0$; density at time t is $v_i/w_i - \lambda_i t$; capacity W. Choose the schedule and fractions that maximise the total value.

Input representation

Array of structs `{id, w, d = v/w, lambda}`. A schedule is a sequence of time slots 0..W-1; each slot holds one unit of some item. A DP table `dp[i][k]` = best value when the first i items (in sorted order) occupy exactly the first k slots, plus `choice[i][k]` for reconstruction.

Algorithm and correctness

- **Exchange argument (ordering).** Put a unit of item i in slot t1 and a unit of item j in slot t2 > t1. Swapping them changes the value by $(\lambda_i - \lambda_j)(t2 - t1)$. So the faster-decaying item must come first: in an optimal schedule items are consumed in **non-increasing order of** λ, each as one contiguous block. The decay rates, not the starting densities, decide the order.
- **Why plain 'highest current density first' fails.** Taking the best-looking unit now ignores that slow-decaying items can wait. On the sample, that naive rule gives 19 while the optimum is 27 (printed by the program).
- **Choosing the fractions.** After sorting, only the amounts x_i ($0 <= x_i <= w_i$) remain. If an item starts in slot s and takes x units, its value is $x \cdot d_i - \lambda_i(x \cdot s + x(x-1)/2)$. Because the blocks are contiguous, dp[i][k] = max over x of dp[i-1][k-x] + gain(i, k-x, x). Skipping an item is x = 0, and the answer is $\max_{k<=W}$ dp[n][k] (stopping early is allowed, so negative-density units are never forced).
- If all $\lambda_i = 0$ the problem collapses to the classical fractional knapsack: sort by v/w and fill greedily in O(n log n).

Complexity analysis

- Sorting: O(n log n).
- DP: n items × (W+1) states × at most $\min(w_i, W)+1$ transitions = $O(n \cdot W \cdot w_{max})$ time, i.e. $O(n W^2)$ in the worst case.
- Space: O(nW) to reconstruct the schedule, O(W) if only the optimal value is needed.

C program

```
/* Q1: Fractional Knapsack with Deterioration Rate
 * Model: weight is consumed in unit "slots" (1 slot = 1 time unit = 1 unit of weight).
 * A unit of item i consumed in slot t yields  d_i - lambda_i * t,  d_i = v_i / w_i.
 * Fractions of an item are allowed (any x_i in [0, w_i]).
 * Algorithm: sort by lambda DESC (exchange argument), then DP over "slots used so far".
 * Input : n W  then n lines: v_i w_i lambda_i        */
#include <stdio.h>
#include <stdlib.h>

typedef struct { int id, w; double d, lam; } Item;

static int cmp(const void *a, const void *b) {          /* lambda desc, then density desc */
    const Item *x = a, *y = b;
    if (x->lam != y->lam) return (x->lam > y->lam) ? -1 : 1;
    if (x->d   != y->d)   return (x->d   > y->d)   ? -1 : 1;
    return 0;
}

/* value of putting x units of item it into slots s, s+1, ..., s+x-1 */
static double gain(const Item *it, int s, int x) {
    return x * it->d - it->lam * ((double)x * s + x * (x - 1) / 2.0);
}

/* naive "pick highest current density each slot" - shown only to prove it is NOT optimal */
static double naive(Item *a, int n, int W) {
    int *left = malloc(n * sizeof(int)); double tot = 0;
    for (int i = 0; i < n; i++) left[i] = a[i].w;
    for (int t = 0; t < W; t++) {
        int b = -1; double bv = 0;
        for (int i = 0; i < n; i++)
            if (left[i] > 0 && a[i].d - a[i].lam * t > bv) { bv = a[i].d - a[i].lam * t; b = i; }
        if (b < 0) break;
        left[b]--; tot += bv;
    }
    free(left); return tot;
}

int main(void) {
    int n, W;
    if (scanf("%d %d", &n, &W) != 2) return 1;
    Item *a = malloc(n * sizeof(Item));
    for (int i = 0; i < n; i++) {
        double v; int w; double lam;
        scanf("%lf %d %lf", &v, &w, &lam);
        a[i].id = i + 1; a[i].w = w; a[i].d = v / w; a[i].lam = lam;
    }
    double nv = naive(a, n, W);
    qsort(a, n, sizeof(Item), cmp);                     /* O(n log n) */

    const double NEG = -1e18;
    double *prev = malloc((W + 1) * sizeof(double)), *cur = malloc((W + 1) * sizeof(double));
    int **ch = malloc((n + 1) * sizeof(int *));
    for (int i = 0; i <= n; i++) ch[i] = calloc(W + 1, sizeof(int));
    for (int k = 0; k <= W; k++) prev[k] = (k == 0) ? 0 : NEG;

    for (int i = 1; i <= n; i++) {                      /* O(n * W * min(w_i, W)) */
        for (int k = 0; k <= W; k++) {
            cur[k] = NEG; ch[i][k] = 0;
            for (int x = 0; x <= a[i-1].w && x <= k; x++) {
                if (prev[k - x] <= NEG / 2) continue;
                double val = prev[k - x] + gain(&a[i-1], k - x, x);
                if (val > cur[k]) { cur[k] = val; ch[i][k] = x; }
            }
        }
        double *t = prev; prev = cur; cur = t;
    }
    int bk = 0;
    for (int k = 0; k <= W; k++) if (prev[k] > prev[bk]) bk = k;
    double best = prev[bk];

    int *take = calloc(n + 1, sizeof(int)), *start = calloc(n + 1, sizeof(int));
    for (int i = n, k = bk; i >= 1; i--) { take[i] = ch[i][k]; k -= take[i]; start[i] = k; }

    printf("Optimal total value = %.4f   (naive highest-density-first = %.4f)\n", best, nv);
    printf("Schedule (time slots 0..%d):\n", bk - 1);
    double chk = 0;
    for (int i = 1; i <= n; i++) {
        if (take[i] == 0) continue;
        printf("  item %d : take %d of %d units (fraction %.3f), slots %d..%d\n",
               a[i-1].id, take[i], a[i-1].w, (double)take[i] / a[i-1].w,
               start[i], start[i] + take[i] - 1);
        for (int t = start[i]; t < start[i] + take[i]; t++) chk += a[i-1].d - a[i-1].lam * t;
    }
    printf("Re-simulated value  = %.4f  -> %s\n", chk, (chk - best < 1e-9 && best - chk < 1e-9) ? "consistent" : "MISMATCH");
    return 0;
}
```

Sample run(s)

Input (n=4 items, W=8; each line is v w lambda):

```
4 8
10 1 0
9 1 5
12 4 0.5
20 5 1
```

Output:

```
Optimal total value = 27.0000   (naive highest-density-first = 19.0000)
Schedule (time slots 0..6):
  item 2 : take 1 of 1 units (fraction 1.000), slots 0..0
  item 4 : take 2 of 5 units (fraction 0.400), slots 1..2
  item 3 : take 3 of 4 units (fraction 0.750), slots 3..5
  item 1 : take 1 of 1 units (fraction 1.000), slots 6..6
Re-simulated value  = 27.0000  -> consistent
```

Q2. Huffman Coding and the Canonical Codebook

Problem. Given symbol frequencies, build a prefix-free binary code with minimum expected length and print the canonical codebook (codes ordered by length, then lexicographically).

Input representation

Arrays `freq[0..2n-2]` and `parent[]` describe the tree (leaves are ids 0..n-1, merged nodes get larger ids). A binary min-heap of node ids ordered by (frequency, id) drives the construction; the id tie-break makes the output deterministic.

Algorithm and correctness

- **Greedy choice:** the two least frequent symbols can be made siblings at the deepest level of some optimal tree (swap argument). **Optimal substructure:** replacing them by one node of weight f_a+f_b gives a smaller instance whose optimal cost differs by exactly f_a+f_b. Repeating n-1 times gives an optimal tree.
- Code length of a symbol = its depth. Because parents have larger ids than children, all depths are computed in one reverse sweep, O(n). A single symbol gets length 1.
- **Canonical codes:** sort symbols by (length, symbol). The first gets all zeros; each next code is (previous code + 1) followed by zeros until the new length is reached, i.e. (prev+1) << (len_{new} - len_{prev}). Lengths are unchanged (so the expected length is still minimal) and the result is prefix-free because the lengths satisfy Kraft's equality.

Complexity analysis

- Heap construction + (n-1) merges, each with 2 pops and 1 push: O(n log n).
- Depth computation O(n); sorting O(n log n); writing the codewords O(n·L) where L <= n-1 is the longest code.
- Total O(n log n + nL), space O(n + nL). The program verifies prefix-freeness and that sum($f_i \cdot len_i$) equals the sum of all merged weights.

C program

```
/* Q2: Huffman Coding + canonical codebook
 * Input : n  then n lines:  symbol(char)  frequency
 * Representation: arrays freq[], parent[] of size 2n-1 + binary min-heap of node ids
 * (key = (frequency, node id) so ties are broken deterministically).            */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 256

static long long fr[2 * MAXN];
static int parent[2 * MAXN], depth[2 * MAXN], heap[2 * MAXN], hs = 0;

static int less(int a, int b) { return fr[a] < fr[b] || (fr[a] == fr[b] && a < b); }
static void push(int x) {
    int i = hs++; heap[i] = x;
    while (i > 0 && less(heap[i], heap[(i - 1) / 2])) {
        int p = (i - 1) / 2, t = heap[p]; heap[p] = heap[i]; heap[i] = t; i = p;
    }
}
static int pop(void) {
    int top = heap[0]; heap[0] = heap[--hs];
    int i = 0;
    for (;;) {
        int l = 2 * i + 1, r = l + 1, m = i;
        if (l < hs && less(heap[l], heap[m])) m = l;
        if (r < hs && less(heap[r], heap[m])) m = r;
        if (m == i) break;
        int t = heap[m]; heap[m] = heap[i]; heap[i] = t; i = m;
    }
    return top;
}

static char sym[MAXN];
static int len[MAXN], ord[MAXN];
static int cmp(const void *a, const void *b) {         /* by (length, symbol) */
    int x = *(const int *)a, y = *(const int *)b;
    if (len[x] != len[y]) return len[x] - len[y];
    return (unsigned char)sym[x] - (unsigned char)sym[y];
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n < 1 || n > MAXN) return 1;
    for (int i = 0; i < n; i++) {
        if (scanf(" %c %lld", &sym[i], &fr[i]) != 2) return 1;
    }
    long long total = 0; for (int i = 0; i < n; i++) total += fr[i];

    int next = n; long long mergedSum = 0;
    for (int i = 0; i < n; i++) push(i);                /* O(n) heapify-by-push: O(n log n) */
    while (hs > 1) {                                    /* n-1 merges, each O(log n) */
        int a = pop(), b = pop();
        fr[next] = fr[a] + fr[b]; mergedSum += fr[next];
        parent[a] = parent[b] = next; push(next++);
    }
    int root = next - 1;
    depth[root] = 0;                                    /* parents always have larger ids */
    for (int v = root - 1; v >= 0; v--) depth[v] = depth[parent[v]] + 1;
```

```
    for (int i = 0; i < n; i++) len[i] = (n == 1) ? 1 : depth[i];

    for (int i = 0; i < n; i++) ord[i] = i;
    qsort(ord, n, sizeof(int), cmp);                    /* O(n log n) */

    static char code[MAXN][MAXN + 1];
    char cur[MAXN + 1]; int cl = 0;
    for (int k = 0; k < n; k++) {                       /* canonical assignment */
        int s = ord[k];
        if (k == 0) { cl = len[s]; memset(cur, '0', cl); }
        else {
            for (int p = cl - 1; p >= 0; p--) {         /* binary +1 */
                if (cur[p] == '0') { cur[p] = '1'; break; } else cur[p] = '0';
            }
            while (cl < len[s]) cur[cl++] = '0';
        }
        memcpy(code[s], cur, cl); code[s][cl] = 0;
    }

    printf("Canonical Huffman codebook:\n");
    long long bits = 0;
    for (int k = 0; k < n; k++) {
        int s = ord[k];
        printf("  '%c'  freq=%-6lld len=%-2d code=%s\n", sym[s], fr[s], len[s], code[s]);
        bits += fr[s] * len[s];
    }
    printf("Total bits = %lld, expected length = %.4f bits/symbol\n", bits, (double)bits / total);

    int ok = 1;                                         /* validation */
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
        if (i != j && strncmp(code[i], code[j], len[i]) == 0 && len[i] <= len[j]) ok = 0;
    printf("Prefix-free check: %s\n", ok ? "PASS" : "FAIL");
    if (n > 1) printf("Cost == sum of merged weights (%lld): %s\n", mergedSum, mergedSum == bits ? "PASS" : "FAIL");
    return 0;
}
```

Sample run(s)

Input (n=6 symbols; each line is symbol frequency):

```
6
a 45
b 13
c 12
d 16
e 9
f 5
```

Output:

```
Canonical Huffman codebook:
  'a'  freq=45     len=1  code=0
  'b'  freq=13     len=3  code=100
  'c'  freq=12     len=3  code=101
  'd'  freq=16     len=3  code=110
  'e'  freq=9      len=4  code=1110
  'f'  freq=5      len=4  code=1111
Total bits = 224, expected length = 2.2400 bits/symbol
Prefix-free check: PASS
Cost == sum of merged weights (224): PASS
```

Q3. Minimum Initial Fuel / Minimum Refuelling Stops (Reverse Greedy)

Problem. A vehicle starts with fuel F and must reach distance D; station i is at distance d_i and supplies f_i fuel. Find the minimum number of refuelling stops.

Input representation

Stations stored in an array of `(d, f)` pairs sorted by distance, plus a binary **max-heap** holding the fuel amounts of stations that have already been passed. The current *reach* = F + (fuel taken so far).

Algorithm and correctness

- **Reverse (retroactive) greedy:** drive as far as the current reach allows, pushing every station passed into the max-heap *without* deciding anything. Only when we cannot reach the target do we decide, in hindsight, that we had stopped at the richest station passed so far (pop the maximum). Repeat until reach >= D; if the heap is empty we are stuck and the answer is -1.
- **Correctness (exchange argument):** any plan with k stops reaches at most F + the sum of its k fuel amounts, and those stations must all be within the reach at the time. Using the k largest stations available at each moment never reduces the reach, so greedy needs no more stops than any other plan.
- **Minimum initial fuel:** the number of stops needed is non-increasing in F, so the smallest F that allows at most K stops is found by binary search on F in [0, D] using the same routine (printed by the program).

Complexity analysis

- Sorting O(n log n). Each station is pushed once and popped at most once: O(n log n) for the greedy.
- Minimum initial fuel for a given K: O(n log n · log D). Space O(n).

C program

```
/* Q3: Minimum refuelling stops (reverse / retroactive greedy)
 * Fuel use = 1 per unit distance.  Input: D F n  then n lines: d_i f_i
 * Representation: array of stations sorted by distance + max-heap of fuel amounts. */
#include <stdio.h>
#include <stdlib.h>

typedef struct { long long d, f; } Station;
static int cmp(const void *a, const void *b) {
    long long x = ((const Station *)a)->d, y = ((const Station *)b)->d;
    return (x > y) - (x < y);
}

static long long *hp; static int hs;
static void hpush(long long v) {
    int i = hs++; hp[i] = v;
    while (i > 0 && hp[(i - 1) / 2] < hp[i]) {
        long long t = hp[i]; hp[i] = hp[(i - 1) / 2]; hp[(i - 1) / 2] = t; i = (i - 1) / 2;
    }
}
static long long hpop(void) {
    long long top = hp[0]; hp[0] = hp[--hs];
    int i = 0;
    for (;;) {
        int l = 2 * i + 1, r = l + 1, m = i;
        if (l < hs && hp[l] > hp[m]) m = l;
        if (r < hs && hp[r] > hp[m]) m = r;
        if (m == i) break;
        long long t = hp[m]; hp[m] = hp[i]; hp[i] = t; i = m;
    }
    return top;
}

/* returns min #stops or -1.  Stations must already be sorted.  O(n log n) */
static int minStops(long long D, long long F, const Station *s, int n) {
    hs = 0; long long reach = F; int i = 0, stops = 0;
    while (reach < D) {
        while (i < n && s[i].d <= reach) hpush(s[i++].f);   /* stations we have passed */
        if (hs == 0) return -1;                             /* stuck */
        reach += hpop();                                    /* retroactively refuel at richest one */
        stops++;
    }
    return stops;
}

int main(void) {
    long long D, F; int n;
    if (scanf("%lld %lld %d", &D, &F, &n) != 3) return 1;
    Station *s = malloc(n * sizeof(Station)); hp = malloc((n + 1) * sizeof(long long));
    for (int i = 0; i < n; i++) if (scanf("%lld %lld", &s[i].d, &s[i].f) != 2) return 1;
    qsort(s, n, sizeof(Station), cmp);

    int r = minStops(D, F, s, n);
    if (r < 0) printf("Target cannot be reached with F = %lld\n", F);
    else       printf("Minimum refuelling stops with F = %lld : %d\n", F, r);

    /* "Minimum initial fuel" view: smallest F that allows at most K stops (binary search, monotone) */
    for (int K = 0; K <= (n < 3 ? n : 3); K++) {
        long long lo = 0, hi = D;
        while (lo < hi) {
            long long mid = lo + (hi - lo) / 2; int q = minStops(D, mid, s, n);
            if (q >= 0 && q <= K) hi = mid; else lo = mid + 1;
        }
        printf("Minimum initial fuel for at most %d stop(s) : %lld\n", K, lo);
    }

    /* validation: exhaustive search over subsets of stations (n <= 20) */
    if (n <= 20) {
        int best = -1;
        for (int mask = 0; mask < (1 << n); mask++) {
            long long reach = F; int cnt = 0, okk = 1;
            for (int i = 0; i < n; i++) if (mask >> i & 1) {
                if (s[i].d > reach) { okk = 0; break; }
                reach += s[i].f; cnt++;
            }
            if (okk && reach >= D && (best < 0 || cnt < best)) best = cnt;
        }
        printf("Brute-force answer = %d  -> %s\n", best, best == r ? "PASS" : "FAIL");
    }
    return 0;
}
```

Sample run(s)

Input (D=100, F=10, five stations; each line is distance fuel):

```
100 10 5
10 60
20 30
30 30
60 40
80 50
```

Output:

```
Minimum refuelling stops with F = 10 : 2
Minimum initial fuel for at most 0 stop(s) : 100
Minimum initial fuel for at most 1 stop(s) : 40
Minimum initial fuel for at most 2 stop(s) : 10
Minimum initial fuel for at most 3 stop(s) : 10
Brute-force answer = 2  -> PASS
```

Q4. Minimum Cost to Connect Sticks

Problem. Joining sticks of lengths x and y costs x+y and produces a stick of length x+y. Find the minimum total cost of joining all sticks into one.

Input representation

Binary min-heap of `long long` lengths (64-bit, since the total can exceed 32 bits).

Algorithm and correctness

- Always join the **two shortest** sticks. This is exactly Huffman merging: the total cost equals the sum over sticks of L_i × (number of times stick i is part of a join), i.e. L_i × depth of its leaf in the merge tree. The shortest two sticks can be placed at the deepest level of an optimal merge tree (swap argument), and the reduced instance has cost lower by x+y, so greedy is optimal.
- The program also contains an exhaustive checker that tries every pair at every step (n <= 8) and prints PASS or FAIL.

Complexity analysis

- n pushes and n-1 rounds of (2 pops + 1 push), each O(log n): O(n log n) time, O(n) space.

C program

```
/* Q4: Minimum cost to connect sticks (Huffman-style merging)
 * Input: n then n lengths.  Representation: binary min-heap of long long. */
#include <stdio.h>
#include <stdlib.h>

static long long *h; static int hs;
static void push(long long v) {
    int i = hs++; h[i] = v;
    while (i > 0 && h[(i - 1) / 2] > h[i]) { long long t = h[i]; h[i] = h[(i-1)/2]; h[(i-1)/2] = t; i = (i-1)/2; }
}
static long long pop(void) {
    long long top = h[0]; h[0] = h[--hs]; int i = 0;
    for (;;) {
        int l = 2*i+1, r = l+1, m = i;
        if (l < hs && h[l] < h[m]) m = l;
        if (r < hs && h[r] < h[m]) m = r;
        if (m == i) break;
        long long t = h[m]; h[m] = h[i]; h[i] = t; i = m;
    }
    return top;
}

static long long greedy(const long long *L, int n) {
    hs = 0; long long cost = 0;
    for (int i = 0; i < n; i++) push(L[i]);
    while (hs > 1) { long long a = pop(), b = pop(); cost += a + b; push(a + b); }
    return cost;
}

/* exhaustive validator: try every pair at every step (n <= 8) */
static long long brute(long long *a, int n) {
    if (n == 1) return 0;
    long long best = -1;
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) {
        long long b[16]; int m = 0;
        for (int k = 0; k < n; k++) if (k != i && k != j) b[m++] = a[k];
        b[m++] = a[i] + a[j];
        long long c = a[i] + a[j] + brute(b, m);
        if (best < 0 || c < best) best = c;
    }
    return best;
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    long long *L = malloc(n * sizeof(long long)); h = malloc((n + 1) * sizeof(long long));
    for (int i = 0; i < n; i++) if (scanf("%lld", &L[i]) != 1) return 1;
    long long g = greedy(L, n);
    printf("Minimum total cost = %lld\n", g);
    if (n <= 8) { long long b = brute(L, n); printf("Exhaustive check   = %lld -> %s\n", b, b == g ? "PASS" : "FAIL"); }
    return 0;
}
```

Sample run(s)

Input (5 sticks):

```
5
2 4 3 6 1
```

Output:

```
Minimum total cost = 35
Exhaustive check   = 35 -> PASS
```

Q5. Candy Distribution (Bi-directional Slope Greedy)

Problem. n children in a line with ratings; each gets at least one candy and a child with a higher rating than an immediate neighbour gets more candy than that neighbour. Minimise the total.

Input representation

Integer arrays `rating[]` and `candy[]`. A second method uses only four counters (up, down, peak, total).

Algorithm and correctness

- **Two-pass greedy.** Pass 1 (left to right): if rating[i] > rating[i-1] then candy[i] = candy[i-1]+1. Pass 2 (right to left): if rating[i] > rating[i+1] then candy[i] = max(candy[i], candy[i+1]+1). Pass 1 is the smallest assignment satisfying all left-neighbour constraints, pass 2 raises values only where a right-neighbour constraint demands it; since pass 2 never breaks a left constraint (it only increases a value) and every raise is forced, the result is the pointwise minimum.
- **Slope method (O(1) space).** The rating array is a sequence of ascending runs, descending runs and plateaus. An ascent of length u contributes 1+2+...+u, a descent of length d contributes 1+2+...+d, and the peak is counted once, as the larger of the two runs. The program updates this incrementally and prints both answers so they can be compared.

Complexity analysis

- Both methods make a constant number of linear scans: O(n) time.
- Two-pass uses O(n) extra space; slope method uses O(1) extra space.

C program

```
/* Q5: Candy distribution.  Input: n then n ratings.
 * Method A: two-pass (left-to-right, right-to-left), O(n) time, O(n) space.
 * Method B: slope counting (up / down / peak), O(n) time, O(1) extra space.   */
#include <stdio.h>
#include <stdlib.h>

static long long twoPass(const int *r, int n, int *candy) {
    for (int i = 0; i < n; i++) candy[i] = 1;
    for (int i = 1; i < n; i++)                       /* left neighbour constraint */
        if (r[i] > r[i-1]) candy[i] = candy[i-1] + 1;
    for (int i = n - 2; i >= 0; i--)                  /* right neighbour constraint */
        if (r[i] > r[i+1] && candy[i] <= candy[i+1]) candy[i] = candy[i+1] + 1;
    long long s = 0; for (int i = 0; i < n; i++) s += candy[i];
    return s;
}

static long long slope(const int *r, int n) {
    if (n <= 1) return n;
    long long total = 1; int up = 0, down = 0, peak = 0;
    for (int i = 1; i < n; i++) {
        if (r[i] > r[i-1])       { up++; down = 0; peak = up; total += 1 + up; }
        else if (r[i] == r[i-1]) { up = down = peak = 0;      total += 1; }
        else { up = 0; down++; total += 1 + down; if (peak >= down) total--; }
    }
    return total;
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int *r = malloc(n * sizeof(int)), *c = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) if (scanf("%d", &r[i]) != 1) return 1;
    long long a = twoPass(r, n, c), b = slope(r, n);
    printf("Candies per child:");
    for (int i = 0; i < n; i++) printf(" %d", c[i]);
    printf("\nMinimum total candies (two-pass) = %lld\n", a);
    printf("Minimum total candies (slope)    = %lld  -> %s\n", b, a == b ? "methods agree" : "MISMATCH");
    return 0;
}
```

Sample run(s)

Input (7 ratings):

```
7
1 0 2 2 3 1 0
```

Output:

```
Candies per child: 2 1 2 1 3 2 1
Minimum total candies (two-pass) = 12
Minimum total candies (slope)    = 12  -> methods agree
```

Q6. Reorganise String with K-Distance Apart

Problem. Rearrange S so that equal characters are at least K positions apart; return the empty string if impossible.

Input representation

Two arrays indexed by character code: `cnt[256]` (copies still unplaced) and `last[256]` (position where the character was last placed). A character is *available* at position p when p - last[c] >= K.

Algorithm and correctness

- **Greedy:** at every position place the available character with the most copies left (ties: the one used longest ago). If no character is available, the answer is the empty string. K <= 1 needs no change.
- **Why it is correct:** the character with most copies left is the bottleneck - delaying it can only make later positions harder (exchange argument), and the tie-break keeps characters cycling in a fixed round-robin order, which is the densest legal pattern. A necessary condition is (maxFreq - 1)·K + (number of characters with maxFreq) <= n; the greedy fails exactly when this cannot be met.
- The program verifies its answer (same multiset of characters, no equal pair closer than K).

Complexity analysis

- n positions, each scans the alphabet (sigma = 256): O(n·sigma) = O(n) for a fixed alphabet. Replacing the scan by a max-heap plus a FIFO cool-down queue gives O(n log sigma).
- Space O(n + sigma).

C program

```
/* Q6: Rearrange string so equal characters are >= K apart.
 * Input: K then the string S (no spaces).  Output: rearranged string or "" (empty).
 * Representation: count[256] and last[256] (last position used).
 * Greedy: at each position choose, among characters whose cool-down has expired,
 *         the one with the most remaining copies (tie -> used longest ago).     */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static char *reorganize(const char *S, int K) {
    int n = (int)strlen(S);
    char *out = malloc(n + 1);
    if (K <= 1) { strcpy(out, S); return out; }
    int cnt[256] = {0}, last[256];
    for (int c = 0; c < 256; c++) last[c] = -1000000000;
    for (int i = 0; i < n; i++) cnt[(unsigned char)S[i]]++;
    for (int pos = 0; pos < n; pos++) {                /* n steps, each scans sigma = 256 */
        int best = -1;
        for (int c = 0; c < 256; c++) {
            if (cnt[c] == 0 || pos - last[c] < K) continue;
            if (best < 0 || cnt[c] > cnt[best] || (cnt[c] == cnt[best] && last[c] < last[best]))
                best = c;
        }
        if (best < 0) { out[0] = 0; return out; }      /* impossible */
        out[pos] = (char)best; cnt[best]--; last[best] = pos;
    }
    out[n] = 0; return out;
}

static int valid(const char *S, const char *T, int K) {  /* validator */
    int n = (int)strlen(S);
    if ((int)strlen(T) != n) return 0;
    int a[256] = {0}, b[256] = {0};
    for (int i = 0; i < n; i++) { a[(unsigned char)S[i]]++; b[(unsigned char)T[i]]++; }
    if (memcmp(a, b, sizeof a)) return 0;
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n && j - i < K; j++) if (T[i] == T[j]) return 0;
    return 1;
}

int main(void) {
    int K; char S[100005];
    if (scanf("%d %100000s", &K, S) != 2) return 1;
    char *T = reorganize(S, K);
    if (T[0] == 0 && S[0] != 0) printf("Result: \"\" (impossible)\n");
    else printf("Result: %s   [validation: %s]\n", T, valid(S, T, K) ? "PASS" : "FAIL");
    return 0;
}
```

Sample run(s)

Input (K=3, S=aabbcc):

```
3 aabbcc
```

Output:

```
Result: abcabc   [validation: PASS]
```

Input (K=3, S=aaabc (impossible)):

```
3 aaabc
```

Output:

```
Result: "" (impossible)
```

Input (K=2, S=aaadbbcc):

```
2 aaadbbcc
```

Output:

```
Result: abcadbca   [validation: PASS]
```

Q7. Minimise Deviation in an Array (Two-Way Greedy with Max-Heap)

Problem. Operations: multiply an odd element by 2, or divide an even element by 2, any number of times. Minimise max(A) - min(A).

Input representation

Max-heap of 64-bit integers, plus one variable `mn` holding the current minimum.

Algorithm and correctness

- **Normalise:** double every odd element. An odd number can only be x or 2x, and an even number can reach every value $o \cdot 2^j$ down to its odd part, so after doubling the odds every element sits at its *largest* reachable value and the only useful move is 'divide an even element by 2'.
- **Greedy loop:** pop the maximum; record max - min; if the maximum is odd it cannot be reduced and the answer cannot improve any more - stop. Otherwise halve it, update the minimum and push it back. Only the maximum matters for shrinking the range (reducing any other element can only lower the minimum or leave the maximum unchanged), so halving the maximum is the only move that can help.
- The program compares the result with an exhaustive search over every element's reachable values (n <= 8).

Complexity analysis

- Each element can be halved at most $\log_2(M)$ times, M = largest value after doubling, so there are O(n log M) heap operations of cost O(log n) each: **O(n log n log M)** time, O(n) space.

C program

```
/* Q7: Minimise deviation (max - min) with ops: odd -> *2, even -> /2.
 * Idea: make every element even (odd*2) -> now only "divide even by 2" is useful.
 * Keep a max-heap + running min; repeatedly halve the maximum while it is even.
 * Input: n then n positive integers.                                          */
#include <stdio.h>
#include <stdlib.h>

static long long *h; static int hs;
static void push(long long v) {
    int i = hs++; h[i] = v;
    while (i > 0 && h[(i-1)/2] < h[i]) { long long t = h[i]; h[i] = h[(i-1)/2]; h[(i-1)/2] = t; i = (i-1)/2; }
}
static long long pop(void) {
    long long top = h[0]; h[0] = h[--hs]; int i = 0;
    for (;;) {
        int l = 2*i+1, r = l+1, m = i;
        if (l < hs && h[l] > h[m]) m = l;
        if (r < hs && h[r] > h[m]) m = r;
        if (m == i) break;
        long long t = h[m]; h[m] = h[i]; h[i] = t; i = m;
    }
    return top;
}

static long long minDeviation(const long long *a, int n) {
    hs = 0; long long mn = (1LL << 60);
    for (int i = 0; i < n; i++) {
        long long x = (a[i] % 2) ? a[i] * 2 : a[i];
        push(x); if (x < mn) mn = x;
    }
    long long ans = (1LL << 60);
    for (;;) {
        long long mx = pop();
        if (mx - mn < ans) ans = mx - mn;
        if (mx % 2) break;                 /* max is odd: cannot decrease it any more */
        mx /= 2; if (mx < mn) mn = mx;
        push(mx);
    }
    return ans;
}

/* validator: every element can become o*2^j, 0<=j<=max(e,1) (o odd part, e = #factors of 2) */
static long long BR; static const long long *A; static int N;
static void rec(int i, long long mx, long long mn) {
    if (i == N) { if (mx - mn < BR) BR = mx - mn; return; }
    long long o = A[i]; int e = 0;
    while (o % 2 == 0) { o /= 2; e++; }
    int top = e > 1 ? e : 1;
    for (int j = 0; j <= top; j++) {
        long long v = o << j;
        rec(i + 1, mx > v ? mx : v, mn < v ? mn : v);
    }
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    long long *a = malloc(n * sizeof(long long)); h = malloc((n + 1) * sizeof(long long));
    for (int i = 0; i < n; i++) if (scanf("%lld", &a[i]) != 1) return 1;
    long long r = minDeviation(a, n);
    printf("Minimum deviation = %lld\n", r);
    if (n <= 8) {
        BR = (1LL << 60); A = a; N = n; rec(0, 0, (1LL << 60));
        printf("Exhaustive check  = %lld -> %s\n", BR, BR == r ? "PASS" : "FAIL");
    }
    return 0;
}
```

Sample run(s)

Input (A = [1,2,3,4]):

```
4
1 2 3 4
```

Output:

```
Minimum deviation = 1
Exhaustive check  = 1 -> PASS
```

Input (A = [4,1,5,20,3]):

```
5
4 1 5 20 3
```

Output:

```
Minimum deviation = 3
Exhaustive check  = 3 -> PASS
```

Q8. Minimum Number of Meeting Rooms

Problem. Given intervals $[s_i, e_i)$, find the minimum number of rooms so that no two overlapping meetings share a room.

Input representation

Two separate integer arrays `starts[]` and `ends[]`, each sorted independently (the pairing of a start with its own end is irrelevant for counting rooms).

Algorithm and correctness

- Sweep the starts in increasing order with a pointer j into the sorted ends. If the next start is earlier than the earliest unmatched end, nothing has been freed, so open a new room; otherwise that meeting reuses the room and j advances.
- **Correctness:** the number of rooms needed equals the maximum number of simultaneously running meetings (interval graphs are perfect), and the sweep maintains exactly 'starts so far minus ends so far'. The program checks this against an $O(n^2)$ brute-force count.

Complexity analysis

- Two sorts O(n log n), one linear sweep O(n): O(n log n) time, O(n) space.

C program

```
/* Q8: Minimum number of meeting rooms.  Intervals are half-open [s, e).
 * Input: n then n lines "s e".
 * Representation: two separate arrays starts[] and ends[], sorted independently. */
#include <stdio.h>
#include <stdlib.h>

static int cmp(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b; return (x > y) - (x < y);
}

static int minRooms(int *s, int *e, int n) {
    qsort(s, n, sizeof(int), cmp);                  /* O(n log n) */
    qsort(e, n, sizeof(int), cmp);
    int rooms = 0, j = 0;
    for (int i = 0; i < n; i++) {                   /* O(n) two-pointer sweep */
        if (s[i] < e[j]) rooms++;                   /* nobody has left yet: open a room */
        else j++;                                   /* earliest-ending meeting freed its room */
    }
    return rooms;
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    int *s = malloc(n * sizeof(int)), *e = malloc(n * sizeof(int));
    int *s0 = malloc(n * sizeof(int)), *e0 = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d %d", &s[i], &e[i]) != 2) return 1;
        s0[i] = s[i]; e0[i] = e[i];
    }
    int r = minRooms(s, e, n);
    printf("Minimum rooms = %d\n", r);
    int mx = 0;                                     /* validator: max overlap at any start time, O(n^2) */
    for (int i = 0; i < n; i++) {
        int c = 0;
        for (int k = 0; k < n; k++) if (s0[k] <= s0[i] && s0[i] < e0[k]) c++;
        if (c > mx) mx = c;
    }
    printf("Max simultaneous meetings (brute force) = %d -> %s\n", mx, mx == r ? "PASS" : "FAIL");
    return 0;
}
```

Sample run(s)

Input (4 meetings; each line is start end):

```
4
0 30
5 10
15 20
10 25
```

Output:

```
Minimum rooms = 3
Max simultaneous meetings (brute force) = 3 -> PASS
```

Q9. Hu-Tucker Greedy Simulation (Optimal Alphabetic Binary Tree)

Problem. Given weights $w_{1..n}$ whose order must be kept in the in-order sequence of leaves, build the binary tree minimising the sum of $w_i \cdot \text{depth}(i)$.

Input representation

Arrays `w[], par[], sq[]` for up to 2n-1 nodes (`sq` = 1 for an original leaf, 0 for a merged node) and a working list `seq[]` of the node ids in current left-to-right order. Phase 3 uses `lc[], rc[]` for the rebuilt ordered tree.

Algorithm and correctness

- **Phase 1 - combination.** Two nodes are *compatible* if no original leaf lies strictly between them. Repeatedly merge the compatible pair of minimum total weight (ties: leftmost); the new node is a non-leaf and takes the place of the left node. This is Huffman's rule restricted by the order constraint: leaves block each other, merged nodes do not. The minimum pair is found in O(m) by a right-to-left scan that keeps the minimum weight in the range up to the next leaf.
- **Phase 2 - levels.** The merge tree itself need not be ordered, but the depth of every original leaf in it is recorded.
- **Phase 3 - recombination.** Scan the leaves left to right with a stack of (node, level); whenever the top two entries have the same level, merge them into one node one level higher. By the Hu-Tucker theorem the level sequence from phase 2 is realisable by an ordered tree, so this ends with a single node whose leaf depths are exactly the phase-2 levels - hence the same optimal cost.
- Plain Huffman ignores the order and gives only a lower bound; Hu-Tucker pays extra only when the order forces it. The program checks the cost against an $O(n^3)$ interval DP (cost[i][j] = $\min_k$ cost[i][k] + cost[k+1][j] + W(i,j)).

Complexity analysis

- Phase 1: n-1 merges, each O(n) here: **$O(n^2)$** time. (With a priority-queue implementation the original algorithm achieves O(n log n); Garsia-Wachs gives the same bound.)
- Phases 2 and 3: O(n) each. Space O(n). The validating DP is $O(n^3)$ time and $O(n^2)$ space.

C program

```
/* Q9: Hu-Tucker algorithm - optimal alphabetic binary tree.
 * Input: n then n weights (in the fixed leaf order).  Minimises sum w_i * depth(i).
 * Phase 1 (combination): repeatedly merge the minimum-weight COMPATIBLE pair
 *         (no original leaf strictly between them); ties -> leftmost; merged node
 *         is a non-leaf and takes the position of the left node.
 * Phase 2 (levels): leaf depths in the merge tree.
 * Phase 3 (recombination): rebuild an ORDERED tree having exactly those depths (stack).
 * Each phase-1 step is found in O(m) with a right-to-left scan -> O(n^2) total.     */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 128
#define INF  (1LL << 60)

static long long w[2 * MAXN]; static int par[2 * MAXN], sq[2 * MAXN], dep[2 * MAXN];
static int lc[2 * MAXN], rc[2 * MAXN];

static void codes(int v, char *buf, int d, int n, char out[][MAXN + 1]) {
    if (v < n) { memcpy(out[v], buf, d); out[v][d] = 0; return; }
    buf[d] = '0'; codes(lc[v], buf, d + 1, n, out);
    buf[d] = '1'; codes(rc[v], buf, d + 1, n, out);
}
static void show(int v, int n) {
    if (v < n) { printf("%d", v + 1); return; }
    printf("("); show(lc[v], n); printf(" "); show(rc[v], n); printf(")");
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n < 1 || n > MAXN) return 1;
    long long W0[MAXN];
    for (int i = 0; i < n; i++) { if (scanf("%lld", &W0[i]) != 1) return 1; w[i] = W0[i]; sq[i] = 1; }
    if (n == 1) { printf("Single leaf, cost 0\n"); return 0; }

    int seq[MAXN], m = n, next = n;
    for (int i = 0; i < n; i++) seq[i] = i;
    while (m > 1) {                                  /* ---- Phase 1: n-1 combinations ---- */
        long long best = INF, runMin = INF; int bi = -1, bj = -1, runArg = -1;
        for (int i = m - 1; i >= 0; i--) {
            int v = seq[i];
            if (runArg >= 0 && w[v] + runMin <= best) { best = w[v] + runMin; bi = i; bj = runArg; }
            if (sq[v]) { runMin = w[v]; runArg = i; }          /* a leaf blocks everything beyond it */
            else if (w[v] <= runMin) { runMin = w[v]; runArg = i; }
        }
        int a = seq[bi], b = seq[bj];
        w[next] = w[a] + w[b]; sq[next] = 0; par[a] = par[b] = next;
        seq[bi] = next++;                             /* merged node replaces the left one */
        for (int k = bj; k < m - 1; k++) seq[k] = seq[k + 1];
        m--;
    }
    int root = next - 1;                              /* ---- Phase 2: levels ---- */
    dep[root] = 0;
    for (int v = root - 1; v >= 0; v--) dep[v] = dep[par[v]] + 1;

    int stN[MAXN], stL[MAXN], sp = 0, nid = n;        /* ---- Phase 3: ordered rebuild ---- */
    for (int i = 0; i < n; i++) {
        stN[sp] = i; stL[sp] = dep[i]; sp++;
        while (sp >= 2 && stL[sp-1] == stL[sp-2]) {
            lc[nid] = stN[sp-2]; rc[nid] = stN[sp-1];
            sp--; stN[sp-1] = nid++; stL[sp-1]--;
        }
    }
    int troot = stN[0];
    int d2[MAXN]; d2[troot] = 0;                       /* final depths from the ordered tree */
    for (int v = nid - 1; v >= n; v--) { d2[lc[v]] = d2[v] + 1; d2[rc[v]] = d2[v] + 1; }
    long long cost = 0;
    for (int i = 0; i < n; i++) cost += W0[i] * d2[i];

    static char cd[MAXN][MAXN + 1]; char buf[MAXN + 1];
    codes(troot, buf, 0, n, cd);
    printf("Optimal alphabetic tree: "); show(troot, n); printf("\n");
    for (int i = 0; i < n; i++) printf("  leaf %d  w=%-4lld depth=%-2d code=%s\n", i + 1, W0[i], d2[i], cd[i]);
    printf("Minimum cost sum(w*depth) = %lld  (stack ended with %d node(s))\n", cost, sp);

    static long long C[MAXN][MAXN], P[MAXN + 1];     /* validator: O(n^3) interval DP */
    P[0] = 0; for (int i = 0; i < n; i++) P[i + 1] = P[i] + W0[i];
    for (int len = 2; len <= n; len++) for (int i = 0; i + len <= n; i++) {
        int j = i + len - 1; long long bst = INF;
        for (int k = i; k < j; k++) if (C[i][k] + C[k+1][j] < bst) bst = C[i][k] + C[k+1][j];
        C[i][j] = bst + P[j + 1] - P[i];
    }
    printf("Interval-DP optimum       = %lld -> %s\n", C[0][n-1], C[0][n-1] == cost ? "PASS" : "FAIL");
    return 0;
}
```

Sample run(s)

Input (6 weights):

```
6
10 1 2 9 3 20
```

Output:

```
Optimal alphabetic tree: (((1 (2 3)) (4 5)) 6)
  leaf 1  w=10   depth=3  code=000
  leaf 2  w=1    depth=4  code=0010
  leaf 3  w=2    depth=4  code=0011
  leaf 4  w=9    depth=3  code=010
  leaf 5  w=3    depth=3  code=011
  leaf 6  w=20   depth=1  code=1
Minimum cost sum(w*depth) = 98  (stack ended with 1 node(s))
Interval-DP optimum        = 98 -> PASS
```

Q10. Greedy Superstring Conjecture (Open Problem)

Problem. Given strings $S = \{s_1,...,s_n\}$, find a shortest string containing every s_i as a substring. This is NP-hard; here we implement the greedy heuristic and measure how far it is from the optimum.

Input representation

Strings as an array of C strings. First duplicates and strings contained in other strings are removed (they never matter), so the remaining set is *substring-free*. overlap(a,b) = length of the longest suffix of a that is also a prefix of b. For n <= 8 the exact optimum is found by trying every order of the strings: the length of an order is the sum of lengths minus the overlaps of consecutive strings.

Algorithm and correctness

- **Greedy:** repeatedly pick the ordered pair (a,b) with the largest overlap, replace them by a + b[overlap:], until one string remains. Output is always a valid superstring (checked by the program).
- **Not optimal in general:** a locally best merge can destroy better overlaps later (see the family S_k below). Whether greedy is always within a factor 2 is the open conjecture discussed next.

Complexity analysis

- Naive implementation above: n rounds × n^2 pairs × $O(L^2)$ per overlap = $O(n^3L^2)$ time (L = maximum length); O(nL) space.
- Standard improvements: overlaps via KMP / prefix functions in O(L) each, or Ukkonen's implementation using Aho-Corasick in O(N) total time-ish for total input length N (plus sorting of overlaps).
- Exact optimum (validation only): $O(n!\cdot n)$, or $O(2^n n^2)$ with a Held-Karp style DP.

What is known about the greedy superstring conjecture

- The shortest common superstring (SCS) problem is NP-hard. The greedy algorithm (merge the pair with maximum overlap, repeat) is simple and works well in practice, e.g. in DNA assembly. Tarhio and Ukkonen conjectured about four decades ago that its output is never longer than twice the optimum.
- The ratio is at least 2: for $S_k = \{c(ab)^k, (ab)^kc, (ba)^k\}$ greedy first merges $c(ab)^k$ with $(ab)^kc$ (overlap 2k) and then has to append $(ba)^k$ with no overlap, giving length 4k+2, while the optimum is 2k+4. The program prints this ratio for growing k; it approaches 2 from below (1.93 at k=40).
- Classical analyses prove only a constant factor larger than 2 for greedy (to my knowledge 3.5 is the best proven upper bound), while the best polynomial-time algorithms achieve about 2.48 (2 11/23) by other techniques.
- **Recent claim.** A preprint by Hiroki Shibata, 'Disproving the Greedy Superstring Conjecture' (arXiv 2609.01365, September 2026), reports instances of equal-length strings (even length k >= 10) on which greedy is at least (9k+2)/(4k+4) times optimal, tending to 9/4 = 2.25, which would refute the 2-approximation conjecture. The author says the counterexample was found with AI assistance. As your problem statement notes, this has not been officially validated: the paper is a preprint and not yet peer reviewed, and I have **not** reproduced its instances here, so treat it as an unconfirmed claim.
- **Practical insight.** Because exact SCS is NP-hard, the programs below only compare greedy with the exact optimum on small inputs (all n! orders of the substring-free set). To explore the open question yourself, feed the program small equal-length strings over a binary alphabet, search for the largest ratio, and note that tie-breaking in greedy is known not to change the worst-case ratio.

C program

```
/* Q10: Greedy shortest-superstring heuristic (+ exact optimum for small n, to measure the ratio)
 * Input: n then n strings (no spaces).  If n == 0 a built-in family is run instead.
 * Representation: array of C strings; overlap ov(a,b) = longest suffix of a that is a prefix of b.
 * Greedy : repeatedly merge the pair with maximum overlap (first pair found on ties).
 * Exact  : try all n! orders of the (substring-free) set - only for n <= 8.        */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXS 12
#define MAXL 512

static int ov(const char *a, const char *b) {
    int la = strlen(a), lb = strlen(b), m = la < lb ? la : lb;
    for (int k = m; k > 0; k--) if (strncmp(a + la - k, b, k) == 0) return k;
    return 0;
}

/* remove duplicates and strings that are substrings of another one; returns new n */
static int reduce(char s[][MAXL], int n) {
    int keep[MAXS] = {0}, m = 0;
    for (int i = 0; i < n; i++) {
        int drop = 0;
        for (int j = 0; j < n && !drop; j++) {
            if (i == j) continue;
            if (strstr(s[j], s[i]) && (strlen(s[j]) > strlen(s[i]) || j < i)) drop = 1;
        }
        keep[i] = !drop;
    }
    for (int i = 0; i < n; i++) if (keep[i]) { if (m != i) strcpy(s[m], s[i]); m++; }
    return m;
}

static void greedy(char s[][MAXL], int n, char *out) {
    while (n > 1) {
        int bi = 0, bj = 1, bo = -1;
        for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int o = ov(s[i], s[j]);
            if (o > bo) { bo = o; bi = i; bj = j; }
        }
        char merged[MAXL]; strcpy(merged, s[bi]); strcat(merged, s[bj] + bo);
        int lo = bi < bj ? bi : bj, hi = bi < bj ? bj : bi;
        strcpy(s[lo], merged);
        strcpy(s[hi], s[n - 1]);
        n--;
    }
    strcpy(out, s[0]);
}

static int OV[MAXS][MAXS], LEN[MAXS], N, bestLen;
static void perm(int *p, int k, int used, int cur) {
    if (k == N) { if (cur < bestLen) bestLen = cur; return; }
    for (int i = 0; i < N; i++) if (!(used >> i & 1)) {
        p[k] = i; perm(p, k + 1, used | 1 << i, k == 0 ? LEN[i] : cur + LEN[i] - OV[p[k-1]][i]);
    }
}
static int optimal(char s[][MAXL], int n) {
    N = n; bestLen = 1 << 30; int p[MAXS];
```

```
    for (int i = 0; i < n; i++) { LEN[i] = strlen(s[i]); for (int j = 0; j < n; j++) OV[i][j] = i == j ? 0 : ov(s[i], s[j]); }
    perm(p, 0, 0, 0);
    return bestLen;
}

static void solve(char in[][MAXL], int n) {
    char s[MAXS][MAXL], g[MAXL], t[MAXS][MAXL];
    for (int i = 0; i < n; i++) strcpy(s[i], in[i]);
    n = reduce(s, n);
    for (int i = 0; i < n; i++) strcpy(t[i], s[i]);
    greedy(s, n, g);
    int ok = 1; for (int i = 0; i < n; i++) if (!strstr(g, t[i])) ok = 0;
    printf("greedy superstring length = %d  (valid superstring: %s)\n", (int)strlen(g), ok ? "yes" : "NO");
    if (n <= 8) {
        int o = optimal(t, n);
        printf("optimal length            = %d\nratio greedy/OPT          = %.3f\n", o, (double)strlen(g) / o);
    }
}

int main(void) {
    int n; static char in[MAXS][MAXL];
    if (scanf("%d", &n) != 1) return 1;
    if (n > 0) {
        for (int i = 0; i < n; i++) if (scanf("%511s", in[i]) != 1) return 1;
        solve(in, n); return 0;
    }
    printf("Family S_k = { c(ab)^k, (ab)^k c, (ba)^k } : ratio tends to 2\n");
    for (int k = 2; k <= 40; k += (k < 8 ? 2 : 8)) {
        char a[MAXL] = "c", b[MAXL] = "", c[MAXL] = "";
        for (int i = 0; i < k; i++) { strcat(a, "ab"); strcat(b, "ab"); strcat(c, "ba"); }
        strcat(b, "c");
        strcpy(in[0], a); strcpy(in[1], b); strcpy(in[2], c);
        printf("k=%-2d: ", k); solve(in, 3);
    }
    return 0;
}
```

Sample run(s)

Input (a 3-string instance):

```
3 cabab ababc baba
```

Output:

```
greedy superstring length = 10  (valid superstring: yes)
optimal length            = 8
ratio greedy/OPT          = 1.250
```

Input (built-in family S_k (n = 0 selects it)):

```
0
```

Output:

```
Family S_k = { c(ab)^k, (ab)^k c, (ba)^k } : ratio tends to 2
k=2 : greedy superstring length = 10  (valid superstring: yes)
optimal length            = 8
ratio greedy/OPT          = 1.250
k=4 : greedy superstring length = 18  (valid superstring: yes)
optimal length            = 12
ratio greedy/OPT          = 1.500
k=6 : greedy superstring length = 26  (valid superstring: yes)
optimal length            = 16
ratio greedy/OPT          = 1.625
k=8 : greedy superstring length = 34  (valid superstring: yes)
optimal length            = 20
ratio greedy/OPT          = 1.700
k=16: greedy superstring length = 66  (valid superstring: yes)
optimal length            = 36
ratio greedy/OPT          = 1.833
k=24: greedy superstring length = 98  (valid superstring: yes)
optimal length            = 52
ratio greedy/OPT          = 1.885
k=32: greedy superstring length = 130  (valid superstring: yes)
optimal length            = 68
ratio greedy/OPT          = 1.912
k=40: greedy superstring length = 162  (valid superstring: yes)
optimal length            = 84
ratio greedy/OPT          = 1.929
```